

Моки внешних ресурсов

Моки методов бизнес-логики, встроенные в код

В платформе реализован механизм моков для вызова методов бизнес-логики без разворота внешних сервисов. Он работает для всех типов вызовов путем перенаправления вызовов в переданную функцию.

Для локальных вызовов нужно создать экземпляр MockService без параметров, для внешних вызовов - передать имя сервиса MockService("online").

Сигнатуры методов должны в точности совпадать с исходными методами, поэтому из Python невозможно замочать методы с оригиналами на C++, которые используют типы, не имеющие представления в Python.

Реализация мок-сервиса расположена в двух отдельных модулях: [MockService](#) (для C++) и [MockService-Py](#) (для Python необходимо подключить оба модуля).

Допустим, у нас есть библиотека my-lib и в ней есть экспортируемая функция MyFunction, в которой выполняется внешний вызов на сервис my-service метода MyObject.MyMethod. Мы хотим протестировать работу этой функции, но без внешних вызовов.

```
1  #include <sbis-bl-core/end_point.hpp>
2  #include <sbis-bl-core/object.hpp>
3  ...
4
5  void MyFunction()
6  {
7
8      ...
9
10     // Создаем endpoint для вызова на внешний сервис
11     sbis::blcore::EndPoint ep( L"my-service" );
12     // Объявляем объект на этом сервисе
13     blcore::Object obj( L"MyObject", ep );
14     // Выполняем вызов метода
15     auto result = obj.Invoke< MyType >( L"MyMethod", Args... ) );
16
17     ...
18
19 }
20
21 ...
```

Для C++ тестов без сервиса



[Простой тест на C++](#)

При написании простого теста, не имеющего внутри себя сервиса, необходимо в коде теста вызвать функцию инициализации модуля BL Core.

1. В файле CMakeLists.txt тестового проекта необходимо добавить зависимость от библиотек sbis-mock-service, sbis-lib300, sbis-bl-core300 и sbis-rpc-service300.

sbis_enable_static_build(GENERATE_STATIC_INITIALIZERS "InitTest") указывает на то, что проект может быть собран со статическими зависимостями и имеет функцию инициализации InitTest, необходимую для инициализации модуля BL Core и работы с системой вызовов методов бизнес-логики.

```
1  cmake_minimum_required(VERSION 3.18.5)
2  include( "$ENV{SBISPlatformSDK_241000}/tools/cmake_support/sbis.cmake"
```

```

)
3 sbis_create_project("test-some-lib" test DEPENDS
4                               boost_system
5                               boost_filesystem
6                               boost_unit_test_framework)
7 sbis_add_sdk_dependencies(
8     sbis-lib300
9     sbis-bl-core300
10    sbis-rpc-service300
11    sbis-mock-service )
12
13 target_sources(${target} PRIVATE
14 "main.cpp"
15 )
16
17 sbis_treat_warnings_as_errors()
18
19 sbis_enable_unity_build()
20
21 sbis_enable_static_build( GENERATE_STATIC_INITIALIZERS INITIALIZERS
22 "InitTest")
23
24 sbis_apply_settings()

```

2. В файле с исходным кодом теста необходимо добавить реализацию функции InitTest и подключить следующие файлы:

```

1 #include <sbis-lib/test.hpp>
2 #include <sbis-lib/boost/filesystem.hpp>
3 #include <sbis-bl-core/init_deinit.hpp>
4 #include <sbis-rpc-service/utils.hpp>
5 #include <sbis-mock-service/mock_service.hpp>
6
7 ...
8 namespace sbis
9 {
10
11 // реализация функции InitTest должна быть расположена в определенном
12 // namespace,
13 // связанным с именем проекта в CMakeLists.txt:
14 // sbis::<имя_проекта>_initializers, в котором '-' заменен на '_'.
15 namespace test_some_lib_initializers
16 {
17 void InitTest()
18 {
19     sbis::rpc::ReadConfigFromIni( boost::none );
20     sbis::bl::InitializeBLCore();
21 }
22 }
23 } // namespace sbis

```

3. В исходном коде теста необходимо настроить RAII-объект mock_service::MockService для создания внешнего сервиса и добавления в него методов.

```

1 namespace {
2 BOOST_AUTO_TEST_SUITE( <strong>main</strong> )
3

```

```

4 BOOST_AUTO_TEST_CASE( TestSomeLib )
5 {
6 // Создание мока сервиса mock_service::MockService( имя
  сервиса(endpoint), на который делается вызов. )
7 auto mock = mock_service::MockService( L"service_name"_sv );
8
9 // Добавление метода mock.AddMethod( ИмяОбъекта.ИмяМетода,
  Ссылка(Указатель) на Функцию или лямбда, реализующая заглушку. )
10 mock.AddMethod( L"MyObject.MyMethod"_sv, [&](String a)-> bool {if( a
  == L"FF" ) return true; return false;} );
11
12 // Вызов тестируемого кода
13
14 ...
15
16 MyFunction();
17
18 ...
19
20 // Автоматическое разрушение при выходе из зоны видимости mock.
21 }
22
23 BOOST_AUTO_TEST_SUITE_END()
24 }

```

Для C++ тестов с сервисом (тесты мобильных микросервисов)



[Подробнее о тесте мобильных микросервисов](#)

1. В s3srv-файл тестового облака необходимо добавить зависимость от модуля MockService с помощью [Genie](#).
2. В файл теста CMakeLists.txt необходимо добавить зависимость от sbis-mock-service.

```

1 cmake_minimum_required(VERSION 3.18.5)
2 include("$ENV{SBISPlatformSDK_241000}/tools/cmake_support/sbis.cmake"
3 )
4 sbis_create_project("test-some-lib" test DEPENDS
5                     boost_system
6                     boost_filesystem
7                     boost_unit_test_framework)
8 sbis_add_sdk_dependencies(
9     sbis-mock-service )
10
11 target_sources(${target} PRIVATE
12 "main.cpp"
13 )
14 sbis_treat_warnings_as_errors()
15
16 sbis_enable_unity_build()
17
18 sbis_apply_settings()

```

3. В коде теста необходимо подключить sbis-mock-service/mock_service.hpp с RAII-объектом mock_service::MockService для создания внешнего сервиса и добавления в него методов.

```

1  #include <sbis-mock-service/mock_service.hpp>
2  ...
3
4  namespace {
5  BOOST_AUTO_TEST_SUITE( <strong>main</strong> )
6
7  BOOST_AUTO_TEST_CASE( TestSomeLib )
8  {
9  // Создание мока сервиса mock_service::MockService( имя
   сервиса(endpoint), на который выполняется вызов. )
10 auto mock = mock_service::MockService( L"service_name"_sv );
11
12 // Добавление метода mock.AddMethod( ИмяОбъекта.ИмяМетода,
   Ссылка(Указатель) на Функцию или лямбда, реализующая заглушку. )
13 mock.AddMethod( L"MyObject.MyMethod"_sv, [&](String a)-> bool {if( a
   == L"FF" ) return true; return false;} );
14
15 // Вызов тестируемого кода
16
17 ...
18
19 MyFunction();
20
21 ...
22
23 // Автоматическое разрушение при выходе из зоны видимости mock.
24 }
25
26 BOOST_AUTO_TEST_SUITE_END()
27 }

```

Для Python-тестов



[Простой тест на Python](#)

1. В s3srv-файл тестового облака необходимо добавить зависимость от модуля MockService и MockService-Py с помощью Genie.
2. В файлах тестов нужно импортировать модуль mock_service.

```

1  import sbis
2  import mock_service
3  import uuid
4  ...
5
6  # Реализация функции, в которую будет перенаправлен вызов.
7  # Функция должна содержать подсказки типов (type_hint) для вывода
   сигнатуры.
8  def MyMockFunction(a:int, b:uuid.UUID)->sbis.Record:
9      ...
10     return sbis.Record()
11
12
13 def TestSomeLib():
14     # создаем объект mock_service

```

```

15     mock = MockService("my-service")
16     # добавляем метод, в качестве функции нельзя передавать лямбду
    выражения
17     mock.AddMethod("MyObject.MyMethod", MyMockFunction)
18
19     # активируем, после активации нельзя добавлять новые методы
20     with mock:
21         ...
22         MyFunction()
23         ...

```

3. Возможные типы для подсказки типов (type_hint)

Python type_hint	Тип в Saby
None	ftUNDEFINED
int	ftINT64
float	ftDOUBLE
str	ftSTRING
bytes	ftBINARY
date	ftDATE
time	ftTIME
datetime	ftDATETIME
timedelta	ftTIMEINTERVAL
Decimal	ftDECIMAL
UUID	ftUUID
dict	ftHASH_TABLE
Record	ftRECORD
RecordSet	ftRECORDSET
list[int]	ftARRAY_INT64
list[float]	ftARRAY_DOUBLE
list[str]	ftARRAY_TEXT
list[bool]	ftARRAY_BOOLEAN
list[date]	ftARRAY_DATE
list[time]	ftARRAY_TIME
list[datetime]	ftARRAY_DATETIME
list[timedelta]	ftARRAY_TIMEINTERVAL
list[Decimal]	ftARRAY_DECIMAL
list[UUID]	ftARRAY_UUID
ObjectId	ftIDENTIFIER
Navigation	ftNAVIGATION
SortingList	ftSORTING
RpcFile	ftRPC_FILE
Flags	ftFLAGS

